SOFTWARE REQUIREMENTS SPECIFICATION

Project Title :

AI-Powered Smart Healthcare Assistant System

Version 1.0

Prepared by:

Anushka Jadhav
Aditi Namde
Vedant Ingle
Shravani Karhale
Harshad Gore
Devesh Kumar

Department Of Computer Engineering

MIT Academy Of Engineering

REVISION HISTORY

Name	**Date**	**Reason for Changes**	**Version**
Anushka Jadhav	March 17, 2026	Initial Document Creation	1.0
Aditi Namde	March 17, 2026	Initial Document Creation	1.0
Vedant Ingle	March 17, 2026	Initial Document Creation	1.0
Shravani Karhale	March 17, 2026	Initial Document Creation	1.0
Harshad Gore	March 17, 2026	Initial Document Creation	1.0
Devesh Kumar	March 17, 2026	Initial Document Creation	1.0

Revision History Table

Contents

1 INTRODUCTION **5**
1.1 Purpose . . . 5
1.2 Document Conventions . . . 5
1.3 Intended Audience and Reading Suggestions . . . 5
1.4 Product Scope . . . 6
1.4.1 Problem Statement . . . 6
1.4.2 Proposed Solution . . . 7
1.4.3 System Goals and Success Metrics . . . 7
1.5 References . . . 8

2 OVERALL DESCRIPTION **9**
2.1 Product Perspective . . . 9
2.1.1 System Architecture . . . 9
2.1.2 External Service Integrations . . . 9
2.2 Product Functions . . . 9
2.3 User Classes and Characteristics . . . 10
2.3.1 Patient (Primary User) . . . 10
2.3.2 Doctor (Professional User) . . . 10
2.3.3 Administrator (System Manager) . . . 10
2.4 Operating Environment . . . 11
2.4.1 Server-Side Stack . . . 11
2.4.2 Client-Side Stack . . . 11
2.4.3 Supported Browsers . . . 11
2.5 Design and Implementation Constraints . . . 11

3 EXTERNAL INTERFACE REQUIREMENTS **13**
3.1 User Interfaces . . . 13
3.1.1 General UI Requirements . . . 13
3.1.2 Patient Interface Components . . . 13
3.1.3 Doctor Interface Components . . . 13
3.1.4 Admin Interface Components . . . 14
3.2 Hardware Interfaces . . . 14
3.3 Software Interfaces . . . 14
3.3.1 Twilio API Interface . . . 14
3.3.2 Vapi API Interface . . . 14
3.3.3 Gemini API Interface . . . 15
3.3.4 n8n Integration . . . 15

4 SYSTEM FEATURES **16**
4.1 Book Appointment . . . 16
4.1.1 Description . . . 16
4.1.2 Stimulus/Response Sequence . . . 16
4.1.3 Functional Requirements . . . 16
4.2 Upload Prescription . . . 17
4.2.1 Description . . . 17
4.2.2 Stimulus/Response Sequence . . . 17

4.2.3 Functional Requirements . . . 17
4.3 Follow Automation Call . . . 17
4.3.1 Description . . . 18
4.3.2 Stimulus/Response Sequence . . . 18
4.3.3 Functional Requirements . . . 18
4.4 Medical History Access . . . 18
4.4.1 Description . . . 18
4.4.2 Stimulus/Response Sequence . . . 19
4.4.3 Functional Requirements . . . 19
4.5 Doctor Verification . . . 19
4.5.1 Description . . . 19
4.5.2 Stimulus/Response Sequence . . . 20
4.5.3 Functional Requirements . . . 20
4.6 AI Chatbot Assistance . . . 20
4.6.1 Description . . . 20
4.6.2 Stimulus/Response Sequence . . . 21
4.6.3 Functional Requirements . . . 21
4.7 User Authentication . . . 21
4.7.1 Description . . . 21
4.7.2 Functional Requirements . . . 22
4.8 Admin Dashboard Management . . . 22
4.8.1 Description . . . 22
4.8.2 Functional Requirements . . . 22

5 OTHER NONFUNCTIONAL REQUIREMENTS 23
5.1 Performance Requirements . . . 23
5.2 Safety Requirements . . . 23
5.3 Security Requirements . . . 23
5.4 Software Quality Attributes . . . 24
5.5 Business Rules . . . 24

6 OTHER REQUIREMENTS 26
6.1 Legal and Regulatory Requirements . . . 26
6.2 Deployment Requirements . . . 26
6.3 Documentation Requirements . . . 26
6.4 Training Requirements . . . 26
6.5 Maintenance Requirements . . . 27

7 APPENDICES 28
7.1 Appendix A: Glossary . . . 28
7.2 Appendix B: Database Schema . . . 28
7.2.1 Users Collection . . . 28
7.2.2 Doctors Collection . . . 29
7.2.3 Appointments Collection . . . 30
7.2.4 Prescriptions Collection . . . 30
7.2.5 MedicalHistory Collection . . . 31
7.2.6 Reminders Collection . . . 31
7.2.7 ChatLogs Collection . . . 32
7.3 Appendix C: API Endpoints . . . 32

7.3.1 Authentication APIs 32
7.3.2 Patient APIs 32
7.3.3 Doctor APIs 33
7.3.4 Appointment APIs 33
7.3.5 Prescription APIs 33
7.3.6 Medical History APIs 34
7.3.7 Admin APIs 34
7.3.8 Chatbot APIs 34
7.3.9 Reminder and Automation APIs 34
7.4 Appendix D: Future Enhancements 35

1 INTRODUCTION

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a comprehensive and detailed description of the requirements for the **AI-Powered Smart Healthcare Assistant System**. This document serves as a complete reference for all stakeholders involved in the development, testing, deployment, and maintenance of the system.

The AI-Powered Smart Healthcare Assistant System is a state-of-the-art web-based platform designed to revolutionize healthcare services by enabling seamless interaction between patients and healthcare providers. The system leverages cutting-edge technologies including React.js for frontend, Node.js for backend, MongoDB for database, and integrates with external services like Twilio, Vapi, Gemini API, and n8n for automation.

1.2 Document Conventions

Convention	Description
REQ-X	Functional requirements identified with prefix "REQ-" followed by unique number
NFR-X	Non-functional requirements identified with prefix "NFR-" followed by unique number
HIGH Priority	Must be implemented in first increment; essential for basic functionality
MEDIUM Priority	Important functionality but can be implemented in later increments
LOW Priority	Optional features that enhance user experience but are not critical

Table 1: Document Conventions

1.3 Intended Audience and Reading Suggestions

Audience	Purpose	Recommended Sections
Project Managers	To understand project scope, timeline, resource allocation	Sections 1, 2, 4, 5
Development Team	To understand technical requirements and implementation	Sections 2.5, 3, 4, 7
Testing/QA Team	To create test cases and validate system functionality	Sections 4, 5, 6
Patients (End Users)	To understand system capabilities and benefits	Sections 1.4, 2.2, 2.3

Audience	Purpose	Recommended Sections
Doctors (End Users)	To understand professional features and responsibilities	Sections 2.3, 4
System Administrators	To understand administrative functions and configuration	Sections 2.3, 4.8, 5
Stakeholders/Investors	To evaluate project viability and business value	Sections 1, 2, 5

1.4 Product Scope

The **AI-Powered Smart Healthcare Assistant System** is a comprehensive web-based healthcare platform designed to bridge the gap between patients and healthcare providers through automation, artificial intelligence, and secure digital services.

1.4.1 Problem Statement

In the current healthcare ecosystem, patients face numerous significant challenges:

Challenges Faced by Patients:

- **Appointment Scheduling Difficulties:** Patients struggle to find available doctors, face long wait times, have no visibility into real-time availability, and cannot easily coordinate appointments with work schedules.
- **Manual Prescription Management:** Paper prescriptions are easily lost, damaged, or forgotten. There is no centralized repository for prescriptions, making it difficult to share with multiple doctors or track medication history.
- **Fragmented Medical History:** Medical records are scattered across multiple healthcare providers with no single source of truth. Patients must remember and repeat medical history at each visit, leading to potential gaps in care.
- **Missed Follow-ups and Medication Adherence:** Up to 50% of patients forget medication schedules, and missed follow-up appointments compromise treatment effectiveness. There are no automated reminders or follow-up systems.
- **Doctor Verification Issues:** Patients have no reliable way to verify doctor credentials, leading to trust and safety concerns. There is no transparency about qualifications and experience.
- **Lack of Immediate Assistance:** Patients need quick answers to healthcare questions but cannot reach medical staff 24/7. Minor health concerns escalate due to lack of timely guidance.

Challenges Faced by Healthcare Providers:

- **Administrative Burden:** Manual appointment management consumes significant staff time. Phone calls for booking, confirmation, and reminders are inefficient and costly.

- **Fragmented Patient Information:** No single view of complete patient medical history. Patients provide incomplete information, and there's no integration with labs and diagnostic centers.
- **Inefficient Communication:** No systematic way to communicate with patients between visits. Difficulty sending reminders and follow-ups at scale.

1.4.2 Proposed Solution

The system addresses these challenges through:

1. **Digital Appointment Management:** Online scheduling with real-time availability, doctor search by specialty and location, automated confirmations, and calendar integration.
2. **Secure Prescription Repository:** Cloud-based storage with encryption, multiple file format support, OCR for text extraction, and easy sharing with doctors.
3. **Centralized Medical History:** Complete health records including diagnoses, treatments, and lab results in chronological order with filtering and export capabilities.
4. **Intelligent Automated Communication:** Twilio for SMS and voice calls, Vapi for AI-powered voice conversations, and automated email notifications.
5. **Doctor Verification System:** Multi-step verification including license validation, certificate verification, and background checks with verified badge display.
6. **AI-Powered Chatbot:** 24/7 assistance using Gemini API for natural language understanding, answering FAQs, and guiding users through the platform.
7. **Workflow Automation:** n8n-powered automation for reminders, follow-ups, and data synchronization.

1.4.3 System Goals and Success Metrics

Goal	Description	Success Metrics
G1: Improve Healthcare Accessibility	Enable patients to book appointments 24/7 from anywhere	50% reduction in scheduling time, 40% increase in online bookings
G2: Enhance Patient Engagement	Improve treatment adherence through automated reminders	30% reduction in missed appointments, 25% increase in medication adherence
G3: Ensure Data Security	Maintain HIPAA and GDPR compliant data storage	Zero security breaches, 100% compliance

Goal	Description	Success Metrics
G4: Build Trust Through Verification	Ensure all doctors are verified and authentic	100% verified doctor profiles, zero complaints about fake doctors
G5: Reduce Administrative Burden	Automate routine tasks to reduce manual work	40% reduction in manual administrative work
G6: Provide 24/7 Instant Support	Offer AI-powered assistance for patient queries	80% query resolution without human intervention

1.5 References

Reference	Description	Version
[1] React.js Documentation	Official documentation for React JavaScript library	18.2.0
[2] Node.js Documentation	Official documentation for Node.js JavaScript runtime	20.x LTS
[3] Express.js Guide	Official guide for Express.js web framework	4.18
[4] MongoDB Manual	Official manual for MongoDB NoSQL database	7.0
[5] Tailwind CSS Documentation	Documentation for Tailwind CSS framework	3.4
[6] JWT.io	JSON Web Token introduction and documentation	RFC 7519
[7] bcrypt Documentation	Documentation for bcrypt password hashing	5.1
[8] Twilio API Documentation	Official documentation for Twilio communication APIs	Latest
[9] Vapi Documentation	Official documentation for Vapi voice AI platform	Latest
[10] Gemini API Documentation	Google's Gemini AI model API documentation	Latest
[11] n8n Documentation	Documentation for n8n workflow automation	Latest
[12] IEEE Std 830-1998	IEEE Recommended Practice for SRS	1998
[13] HIPAA Guidelines	Health Insurance Portability and Accountability Act	2024
[14] GDPR Compliance	General Data Protection Regulation requirements	2024

2 OVERALL DESCRIPTION

2.1 Product Perspective

The system is a standalone product that integrates with several external services to provide comprehensive functionality.

2.1.1 System Architecture

The system follows a modern, scalable three-tier architecture:

- **Presentation Layer:** React.js 18.2 with Tailwind CSS 3.4 for responsive user interfaces
- **Application Layer:** Node.js 20.x LTS with Express.js 4.18 for business logic and APIs
- **Data Layer:** MongoDB 7.0 for primary data storage, Redis 7.2 for caching
- **Integration Layer:** REST APIs connecting to external services

2.1.2 External Service Integrations

- **Twilio API:** SMS notifications, voice calls, two-way messaging
- **Vapi API:** AI-powered voice conversations for intelligent follow-ups
- **Gemini API:** Natural language processing for AI chatbot
- **n8n:** Workflow automation for reminders and data synchronization
- **MongoDB Atlas:** Cloud database with automated backups

2.2 Product Functions

Major functional modules include:

1. **User Authentication and Management:** Registration, login, profile management, password reset, role-based access control
2. **Appointment Booking System:** Doctor search, availability checking, scheduling, rescheduling, cancellations, reminders
3. **Prescription Upload and Management:** File upload, OCR processing, secure storage, easy retrieval, sharing with doctors
4. **Medical History Access:** Chronological records, filtering, exporting, sharing with healthcare providers
5. **Doctor Verification System:** Credential submission, document verification, admin approval, verified badge display
6. **Automated Follow-Up Calls:** Scheduled reminders, AI-powered voice calls, medication adherence tracking

7. **AI Chatbot Assistance:** 24/7 query resolution, appointment guidance, FAQ answering, symptom information
8. **Admin Dashboard Management:** User management, verification queue, system monitoring, report generation

2.3 User Classes and Characteristics

2.3.1 Patient (Primary User)

- **Technical Proficiency:** Basic to intermediate; comfortable with web/mobile apps
- **Usage Pattern:** Occasional (appointments, prescription uploads, history access)
- **Primary Goals:** Book appointments, upload prescriptions, access medical history, receive reminders
- **Age Range:** 18-80 years (wide demographic)
- **Access Method:** Smartphone (primary), tablet, or computer
- **Session Duration:** 5-15 minutes average
- **Peak Usage Times:** Evenings (6-9 PM) and weekends

2.3.2 Doctor (Professional User)

- **Technical Proficiency:** Moderate to high; familiar with practice management software
- **Usage Pattern:** Daily during practice hours (multiple sessions)
- **Primary Goals:** Manage appointments, access patient records, update consultation notes
- **Specialties:** General physicians, cardiologists, dermatologists, etc.
- **Access Method:** Desktop computer (primary), mobile secondary
- **Session Duration:** 15-30 minutes, multiple sessions daily
- **Verification Required:** Medical license, degree certificates, ID proof

2.3.3 Administrator (System Manager)

- **Technical Proficiency:** High; technical background preferred
- **Usage Pattern:** Daily for monitoring and management
- **Primary Goals:** Verify doctors, manage users, monitor system health
- **Role Types:** Super Admin, Verification Admin, Support Admin
- **Access Method:** Desktop computer
- **Authorization Level:** Full system access with role-based restrictions

2.4 Operating Environment

2.4.1 Server-Side Stack

- **Operating System:** Ubuntu Server 22.04 LTS
- **Web Server:** Nginx 1.24 (reverse proxy, load balancing)
- **Runtime Environment:** Node.js 20.x LTS
- **Framework:** Express.js 4.18
- **Database:** MongoDB 7.0 (primary), Redis 7.2 (caching)
- **Process Manager:** PM2 5.3
- **Containerization:** Docker 24.0, Kubernetes 1.28

2.4.2 Client-Side Stack

- **Frontend Framework:** React.js 18.2
- **Styling Framework:** Tailwind CSS 3.4
- **State Management:** Redux Toolkit 2.0
- **HTTP Client:** Axios 1.6
- **Routing:** React Router 6.20
- **Form Handling:** React Hook Form 7.48
- **Validation:** Yup 1.3

2.4.3 Supported Browsers

- Google Chrome 90+
- Mozilla Firefox 88+
- Apple Safari 14+
- Microsoft Edge 90+
- Opera 76+

2.5 Design and Implementation Constraints

ID	Constraint	Description
C1	Frontend Framework	Must use React.js for frontend development
C2	Styling	Must use Tailwind CSS for responsive design

ID	Constraint	Description
C3	Backend Runtime	Must use Node.js for server-side development
C4	Database	Must use MongoDB for primary data storage
C5	Authentication	Must use JWT for stateless authentication
C6	Password Security	Must use bcrypt for password hashing
C7	Communication	Must use Twilio/Vapi for calls and SMS
C8	Automation	Must use n8n for workflow automation
C9	AI Integration	Must use Gemini API for chatbot
C10	API Design	Must follow REST principles
C11	Responsive Design	All interfaces must work on mobile, tablet, desktop
C12	Accessibility	Must comply with WCAG 2.1 AA standards

3 EXTERNAL INTERFACE REQUIREMENTS

3.1 User Interfaces

3.1.1 General UI Requirements

- All interfaces must be responsive across mobile, tablet, and desktop devices
- Consistent color scheme and branding across all pages
- Maximum page load time of 3 seconds
- Support for screen readers (WCAG 2.1 AA compliance)
- Full keyboard navigation support
- Clear error messages with recovery suggestions
- Confirmation dialogs for destructive actions
- Loading indicators for asynchronous operations

3.1.2 Patient Interface Components

- **Login/Registration:** Email/password registration, social login options, password reset
- **Dashboard:** Upcoming appointments, recent prescriptions, quick action buttons
- **Appointment Booking:** Doctor search, specialty filter, date picker, available slots
- **Prescription Upload:** Drag-and-drop interface, file preview, upload progress
- **Medical History:** Chronological timeline, filter options, download buttons
- **Chatbot Widget:** Floating chat button, conversation history, quick replies

3.1.3 Doctor Interface Components

- **Dashboard:** Today's appointments, upcoming schedule, patient search
- **Appointment Management:** Patient details, consultation notes, prescription access
- **Patient Records:** Medical history viewer, prescription viewer, notes editor
- **Profile Management:** Availability settings, verification status, practice information

3.1.4 Admin Interface Components

- **Dashboard:** System statistics, pending verifications, recent activities
- **Verification Queue:** Doctor applications, document viewer, approve/reject buttons
- **User Management:** User list, search/filter, block/unblock actions
- **Reports:** Analytics charts, export options, scheduled reports

3.2 Hardware Interfaces

- **Touchscreen:** Touch events for mobile navigation with gesture support
- **Camera:** MediaDevices API for document scanning and prescription capture
- **Microphone:** Web Audio API for voice input to chatbot
- **Speaker:** Audio API for call notifications and alerts
- **Storage:** LocalStorage/IndexedDB for offline data caching (5-10 MB)

3.3 Software Interfaces

3.3.1 Twilio API Interface

- **Purpose:** Send SMS notifications and make automated voice calls
- **Type:** REST API
- **Protocol:** HTTPS
- **Data Format:** JSON
- **Authentication:** Account SID + Auth Token
- **Endpoints:** /2010-04-01/Accounts/Sid/Messages.json
- **Rate Limits:** Depends on account tier
- **Error Handling:** Retry with exponential backoff (max 3 attempts)

3.3.2 Vapi API Interface

- **Purpose:** AI-powered voice conversations for intelligent follow-ups
- **Type:** REST API + WebSocket
- **Protocol:** HTTPS/WSS
- **Data Format:** JSON
- **Authentication:** API Key
- **Features:** Natural language understanding, voice synthesis, conversation management

3.3.3 Gemini API Interface

- **Purpose:** AI chatbot and natural language processing
- **Type:** REST API
- **Protocol:** HTTPS
- **Data Format:** JSON
- **Authentication:** API Key
- **Rate Limits:** 60 requests per minute (standard tier)
- **Features:** Text generation, question answering, context retention

3.3.4 n8n Integration

- **Purpose:** Workflow automation and task scheduling
- **Type:** Self-hosted or cloud service
- **Protocol:** HTTPS/REST
- **Authentication:** API Key or OAuth
- **Workflows:** Reminder scheduling, data synchronization, report generation

4 SYSTEM FEATURES

4.1 Book Appointment

Priority: HIGH

4.1.1 Description

This feature allows patients to search for doctors by specialty, location, and availability, and book appointments online. It provides real-time slot availability and instant confirmation.

4.1.2 Stimulus/Response Sequence

1. Patient logs into the system and navigates to appointment booking section
2. Patient searches for doctors using filters (specialty, location, experience, ratings)
3. System displays list of available doctors matching criteria
4. Patient selects a doctor and views available time slots
5. Patient chooses preferred date and time slot
6. System checks real-time availability and reserves the slot
7. Patient confirms appointment details
8. System saves appointment to database and sends confirmation
9. Doctor receives notification of new appointment

4.1.3 Functional Requirements

- **REQ-1**: System shall allow patients to search for doctors by specialty, location, and name
- **REQ-2**: System shall display doctor profiles including qualifications, experience, and ratings
- **REQ-3**: System shall show real-time available time slots for selected doctor and date
- **REQ-4**: System shall allow patients to select and confirm appointment time
- **REQ-5**: System shall prevent double-booking by locking slots during selection
- **REQ-6**: System shall send confirmation SMS and email to patient
- **REQ-7**: System shall notify doctor of new appointment via dashboard and notification
- **REQ-8**: System shall allow patients to view their upcoming appointments
- **REQ-9**: System shall allow patients to reschedule or cancel appointments
- **REQ-10**: System shall send reminders 24 hours and 2 hours before appointment

4.2 Upload Prescription

Priority: HIGH

4.2.1 Description

This feature enables patients to upload their prescriptions securely to the cloud, making them accessible for future consultations and sharing with doctors.

4.2.2 Stimulus/Response Sequence

1. Patient navigates to prescription upload section
2. Patient selects file upload option or drags and drops files
3. System validates file format and size
4. System displays preview of uploaded file
5. Patient adds metadata (doctor name, date, prescription type)
6. Patient confirms upload
7. System stores file securely with encryption
8. System processes OCR for text extraction (optional)
9. Upload confirmation is displayed to patient

4.2.3 Functional Requirements

- **REQ-11**: System shall support file upload in JPG, PNG, PDF, and TIFF formats
- **REQ-12**: System shall validate file size (max 10MB per file)
- **REQ-13**: System shall allow drag-and-drop file upload
- **REQ-14**: System shall display upload progress indicator
- **REQ-15**: System shall store files with AES-256 encryption
- **REQ-16**: System shall allow patients to add metadata (doctor, date, notes)
- **REQ-17**: System shall organize prescriptions by date and doctor
- **REQ-18**: System shall allow patients to search and filter prescriptions
- **REQ-19**: System shall allow patients to share prescriptions with doctors
- **REQ-20**: System shall send prescription expiry notifications

4.3 Follow Automation Call

Priority: MEDIUM

4.3.1 Description

This feature provides automated voice calls and reminders for medication adherence, appointment follow-ups, and post-consultation check-ins using Twilio and Vapi.

4.3.2 Stimulus/Response Sequence

1. Scheduled reminder time is reached in n8n workflow
2. n8n triggers automation workflow for patient reminder
3. System retrieves patient contact information and preferences
4. For basic reminders, Twilio initiates automated voice call
5. For intelligent follow-ups, Vapi initiates AI-powered conversation
6. Patient receives call and interacts with voice system
7. Call status and interaction logs are recorded
8. If call fails, system retries up to 3 times with increasing intervals

4.3.3 Functional Requirements

- **REQ-21**: System shall allow patients to set medication reminder schedules
- **REQ-22**: System shall send automated appointment reminders 24 hours before
- **REQ-23**: System shall trigger post-consultation follow-up calls
- **REQ-24**: System shall use Twilio API for basic voice calls and SMS
- **REQ-25**: System shall use Vapi API for AI-powered voice conversations
- **REQ-26**: System shall support multi-language voice prompts
- **REQ-27**: System shall log all call attempts and outcomes
- **REQ-28**: System shall retry failed calls up to 3 times
- **REQ-29**: System shall allow patients to opt out of automated calls
- **REQ-30**: System shall track medication adherence responses

4.4 Medical History Access

Priority: HIGH

4.4.1 Description

This feature provides patients and authorized doctors with secure access to complete medical history including diagnoses, treatments, prescriptions, and lab results.

4.4.2 Stimulus/Response Sequence

1. User selects medical history option from dashboard
2. System verifies user authorization and access permissions
3. System retrieves all medical records for the patient
4. Records are displayed in chronological order with filtering options
5. User can filter records by date range, doctor, or record type
6. User can select specific records for detailed view
7. User can download or share records as needed

4.4.3 Functional Requirements

- **REQ-31**: System shall store all medical records securely with encryption
- **REQ-32**: System shall allow patients to view their complete medical history
- **REQ-33**: System shall allow doctors to access assigned patient medical history
- **REQ-34**: System shall maintain chronological order of all records
- **REQ-35**: System shall allow filtering by date range, doctor, and record type
- **REQ-36**: System shall allow exporting records as PDF
- **REQ-37**: System shall maintain detailed audit log of all record access
- **REQ-38**: System shall allow patients to grant temporary access to new doctors
- **REQ-39**: System shall highlight critical information (allergies, chronic conditions)
- **REQ-40**: System shall support family medical history tracking

4.5 Doctor Verification

Priority: HIGH

4.5.1 Description

This feature ensures all doctors on the platform are properly verified through a multi-step credential validation process before they can receive appointments.

4.5.2 Stimulus/Response Sequence

1. Doctor registers on the platform and provides professional details
2. Doctor uploads required documents (medical license, degree certificates, ID)
3. System stores documents securely and creates verification request
4. Admin receives notification of pending verification
5. Admin reviews documents and validates credentials
6. Admin approves or rejects verification with comments
7. System updates doctor status and sends notification
8. Verified doctors receive verified badge on their profile

4.5.3 Functional Requirements

- **REQ-41**: System shall allow doctor registration with professional details
- **REQ-42**: System shall require upload of medical license and certificates
- **REQ-43**: System shall support document upload in multiple formats
- **REQ-44**: System shall create verification request in admin queue
- **REQ-45**: Admin dashboard shall display pending verifications
- **REQ-46**: Admin shall be able to view and download uploaded documents
- **REQ-47**: Admin shall approve or reject with reason
- **REQ-48**: System shall display verified badge on approved doctor profiles
- **REQ-49**: System shall notify doctors of verification status changes
- **REQ-50**: System shall support periodic reverification (annual)

4.6 AI Chatbot Assistance

Priority: MEDIUM

4.6.1 Description

This feature provides 24/7 AI-powered assistance to patients for answering queries, guiding through the platform, and providing healthcare information using Gemini API.

4.6.2 Stimulus/Response Sequence

1. Patient opens chat widget from any page
2. Patient types or speaks their query
3. System sends query to Gemini API with context
4. AI processes query and generates appropriate response
5. Response is displayed to patient in chat interface
6. Patient can ask follow-up questions
7. Conversation history is maintained for context
8. System logs conversation for quality improvement

4.6.3 Functional Requirements

- **REQ-51**: System shall provide chat interface accessible from all pages
- **REQ-52**: System shall integrate Gemini API for natural language processing
- **REQ-53**: Chatbot shall answer FAQs about appointments and prescriptions
- **REQ-54**: Chatbot shall guide users through platform features
- **REQ-55**: Chatbot shall provide medication information with disclaimers
- **REQ-56**: Chatbot shall include disclaimer about not providing medical advice
- **REQ-57**: System shall maintain conversation context within session
- **REQ-58**: System shall log all chat conversations for improvement
- **REQ-59**: Chatbot shall support escalation to human support
- **REQ-60**: System shall support multi-language capability (future)

4.7 User Authentication

Priority: HIGH

4.7.1 Description

This feature manages user registration, login, password management, and role-based access control for all user types.

4.7.2 Functional Requirements

- **REQ-61**: System shall allow user registration with email and phone number
- **REQ-62**: System shall verify email address via confirmation link
- **REQ-63**: System shall verify phone number via OTP
- **REQ-64**: System shall implement JWT-based authentication
- **REQ-65**: System shall hash passwords using bcrypt with salt rounds
- **REQ-66**: System shall support password reset via email
- **REQ-67**: System shall implement role-based access control (Patient, Doctor, Admin)
- **REQ-68**: System shall have session timeout after 24 hours of inactivity
- **REQ-69**: System shall support "remember me" functionality
- **REQ-70**: System shall log all login attempts for security

4.8 Admin Dashboard Management

Priority: HIGH

4.8.1 Description

This feature provides administrators with comprehensive tools for managing users, verifying doctors, monitoring system health, and generating reports.

4.8.2 Functional Requirements

- **REQ-71**: System shall provide admin dashboard with key statistics
- **REQ-72**: System shall display pending doctor verifications in queue
- **REQ-73**: System shall allow admins to view all registered users
- **REQ-74**: System shall allow admins to search and filter users
- **REQ-75**: System shall allow admins to block/unblock user accounts
- **REQ-76**: System shall allow admins to view system logs
- **REQ-77**: System shall generate reports on appointments and user activity
- **REQ-78**: System shall allow export of reports in CSV and PDF
- **REQ-79**: System shall display system health metrics
- **REQ-80**: System shall allow configuration of system parameters

5 OTHER NONFUNCTIONAL REQUIREMENTS

5.1 Performance Requirements

Requirement ID	Description
NFR-01	System shall respond to user requests within 2 seconds for 95% of requests
NFR-02	Database queries shall complete within 1 second
NFR-03	System shall support 100+ concurrent users
NFR-04	Page load time shall not exceed 3 seconds
NFR-05	System uptime shall be 99.5% excluding scheduled maintenance
NFR-06	API response time shall be under 500ms for 95% of calls
NFR-07	File uploads shall process within 5 seconds for 10MB files
NFR-08	Search results shall display within 2 seconds
NFR-09	System shall handle peak loads of 500 concurrent users
NFR-10	Backup completion shall not exceed 30 minutes

5.2 Safety Requirements

Requirement ID	Description
NFR-11	Medical data shall not be lost or corrupted under any circumstances
NFR-12	Automated daily backups shall be implemented with point-in-time recovery
NFR-13	Only authorized doctors may access patient data
NFR-14	System shall have disaster recovery plan with RTO of 4 hours
NFR-15	Backup data shall be stored in geographically separate location
NFR-16	System shall have redundant power and network infrastructure
NFR-17	Critical operations shall have confirmation dialogs
NFR-18	System shall validate all data before processing
NFR-19	Automatic failover shall be implemented for critical services
NFR-20	Data integrity checks shall run daily

5.3 Security Requirements

Requirement ID	Description
NFR-21	All communications shall use HTTPS/TLS 1.3 encryption
NFR-22	Passwords shall be hashed using bcrypt with minimum 10 salt rounds
NFR-23	JWT tokens shall expire after 24 hours
NFR-24	API endpoints shall implement rate limiting (100 requests/minute)
NFR-25	All access to medical records shall be logged
NFR-26	System shall implement protection against SQL injection and XSS
NFR-27	Session management shall use HTTP-only secure cookies
NFR-28	Failed login attempts shall be limited to 5 per 15 minutes
NFR-29	Data at rest shall be encrypted using AES-256
NFR-30	Regular security audits and penetration testing shall be conducted

5.4 Software Quality Attributes

Attribute	Requirement
Reliability	System uptime shall be 99% availability, with maximum 3 hours downtime per month
Usability	Interface shall be easy for non-technical users (SUS score > 70)
Scalability	System shall support horizontal scaling to 10,000+ users
Maintainability	Code shall follow modular architecture with comprehensive documentation
Portability	System shall run on any modern browser without platform dependencies
Interoperability	System shall integrate with external healthcare systems via HL7/FHIR
Testability	System shall have minimum 80% code coverage
Reusability	Components shall be designed for maximum reusability
Flexibility	System shall support easy addition of new features
Accessibility	System shall comply with WCAG 2.1 AA standards

5.5 Business Rules

Rule ID	Description
BR-01	Only verified doctors can receive appointments from patients
BR-02	Patients can cancel appointments up to 2 hours before scheduled time

Rule ID	Description
BR-03	Admin approval is required for all doctor accounts
BR-04	Medical records must be retained for minimum 7 years
BR-05	User consent is required for data processing and sharing
BR-06	Doctors must renew verification annually
BR-07	Patients can only access their own medical records
BR-08	Doctors can only access assigned patient records
BR-09	All data access must be logged for audit purposes
BR-10	Emergency access override requires admin approval

6 OTHER REQUIREMENTS

6.1 Legal and Regulatory Requirements

- System must comply with HIPAA regulations for healthcare data protection
- System must comply with GDPR for European user data
- System must maintain audit trails for all data access
- System must support data portability requests
- System must implement data deletion on user request

6.2 Deployment Requirements

- Cloud deployment support (AWS, Azure, or GCP)
- Containerization using Docker for consistent environments
- Orchestration using Kubernetes for scalability
- CI/CD pipeline for automated deployment
- Blue-green deployment for zero downtime updates

6.3 Documentation Requirements

- User manual for patients with step-by-step instructions
- User manual for doctors with feature explanations
- Admin guide with configuration details
- API documentation with Swagger/OpenAPI
- Installation guide for deployment
- Troubleshooting guide for common issues

6.4 Training Requirements

- Training materials for administrators
- Video tutorials for patients and doctors
- FAQ section for common queries
- On-site training for clinic staff (optional)
- Webinar sessions for feature updates

6.5 Maintenance Requirements

- Regular security updates and patches
- Database optimization and indexing
- Log rotation and cleanup
- Performance monitoring and tuning
- Backup verification and testing

7 APPENDICES

7.1 Appendix A: Glossary

Term	Definition
API	Application Programming Interface - set of protocols for building software
AI	Artificial Intelligence - simulation of human intelligence in machines
JWT	JSON Web Token - compact URL-safe token for authentication
n8n	Workflow automation tool for connecting services
Twilio	Cloud communications platform for SMS, voice, and video
Vapi	Voice AI platform for automated conversations
Gemini	Google's AI model for natural language processing
HIPAA	Health Insurance Portability and Accountability Act
GDPR	General Data Protection Regulation
OCR	Optical Character Recognition - text extraction from images
REST	Representational State Transfer - architectural style for APIs
MongoDB	NoSQL database for flexible document storage
React	JavaScript library for building user interfaces
Node.js	JavaScript runtime for server-side development
Express	Web framework for Node.js
Tailwind	Utility-first CSS framework

7.2 Appendix B: Database Schema

7.2.1 Users Collection

- **_id:** ObjectId (Primary Key)
- **name:** String (required)
- **email:** String (required, unique)
- **password:** String (required, hashed)
- **phone:** String (required, unique)
- **role:** String (enum: ['patient', 'doctor', 'admin'])
- **profilePicture:** String (URL)
- **dateOfBirth:** Date
- **gender:** String (enum: ['male', 'female', 'other'])

- **address:** Object (street, city, state, zip, country)
- **emergencyContact:** Object (name, phone, relationship)
- **createdAt:** Date
- **updatedAt:** Date
- **lastLogin:** Date
- **isActive:** Boolean (default: true)

7.2.2 Doctors Collection

- **_id:** ObjectId (Primary Key)
- **userId:** ObjectId (reference to Users)
- **specialization:** String (required)
- **qualifications:** Array of Strings
- **experience:** Number (years)
- **licenseNumber:** String (required, unique)
- **licenseExpiry:** Date
- **verificationStatus:** String (enum: ['pending', 'verified', 'rejected'])
- **verificationDate:** Date
- **verifiedBy:** ObjectId (reference to Admin)
- **documents:** Array of Objects (type, url, uploadedAt)
- **consultationFee:** Number
- **availableSlots:** Array of Objects (day, startTime, endTime)
- **hospital:** String
- **rating:** Number (average rating)
- **totalReviews:** Number
- **about:** String
- **languages:** Array of Strings
- **createdAt:** Date
- **updatedAt:** Date

7.2.3 Appointments Collection

- **_id:** ObjectId (Primary Key)
- **patientId:** ObjectId (reference to Users)
- **doctorId:** ObjectId (reference to Doctors)
- **appointmentDate:** Date (required)
- **appointmentTime:** String (required)
- **duration:** Number (minutes, default: 30)
- **status:** String (enum: ['scheduled', 'confirmed', 'completed', 'cancelled', 'no-show'])
- **type:** String (enum: ['in-person', 'video', 'phone'])
- **reason:** String
- **notes:** String (doctor's notes)
- **reminders:** Array of Objects (type, sentAt, status)
- **createdAt:** Date
- **updatedAt:** Date
- **cancelledAt:** Date
- **cancellationReason:** String

7.2.4 Prescriptions Collection

- **_id:** ObjectId (Primary Key)
- **patientId:** ObjectId (reference to Users)
- **doctorId:** ObjectId (reference to Doctors)
- **appointmentId:** ObjectId (reference to Appointments, optional)
- **fileUrl:** String (URL to stored file)
- **fileName:** String
- **fileSize:** Number (bytes)
- **fileType:** String (MIME type)
- **ocrText:** String (extracted text, optional)
- **medications:** Array of Objects (name, dosage, frequency, duration)
- **uploadedAt:** Date
- **expiryDate:** Date
- **tags:** Array of Strings
- **notes:** String

7.2.5 MedicalHistory Collection

- **_id:** ObjectId (Primary Key)
- **patientId:** ObjectId (reference to Users)
- **doctorId:** ObjectId (reference to Doctors)
- **appointmentId:** ObjectId (reference to Appointments)
- **diagnosis:** String
- **symptoms:** Array of Strings
- **treatment:** String
- **medications:** Array of Objects (name, dosage, frequency)
- **labReports:** Array of Strings (URLs)
- **vitals:** Object (bloodPressure, heartRate, temperature, weight)
- **followUpDate:** Date
- **notes:** String
- **createdAt:** Date

7.2.6 Reminders Collection

- **_id:** ObjectId (Primary Key)
- **patientId:** ObjectId (reference to Users)
- **type:** String (enum: ['medication', 'appointment', 'follow-up', 'prescription-refill'])
- **title:** String
- **description:** String
- **scheduledTime:** Date
- **repeat:** String (enum: ['once', 'daily', 'weekly', 'monthly'])
- **status:** String (enum: ['pending', 'sent', 'failed', 'cancelled'])
- **channel:** String (enum: ['sms', 'call', 'email', 'push'])
- **attempts:** Number
- **lastAttempt:** Date
- **response:** String (patient response, optional)
- **createdAt:** Date

7.2.7 ChatLogs Collection

- **_id:** ObjectId (Primary Key)
- **userId:** ObjectId (reference to Users)
- **sessionId:** String
- **query:** String
- **response:** String
- **intent:** String (detected intent)
- **confidence:** Number (0-1)
- **context:** Object
- **timestamp:** Date
- **rating:** Number (user rating, optional)
- **feedback:** String (optional)

7.3 Appendix C: API Endpoints

7.3.1 Authentication APIs

Endpoint	Method	Description
/api/auth/register	POST	Register new user (patient/doctor)
/api/auth/login	POST	User login with email/password
/api/auth/logout	POST	User logout
/api/auth/refresh-token	POST	Refresh JWT token
/api/auth/verify-email/:token	GET	Verify email address
/api/auth/verify-phone	POST	Send phone verification OTP
/api/auth/verify-phone/confirm	POST	Confirm phone OTP
/api/auth/forgot-password	POST	Request password reset
/api/auth/reset-password/:token	POST	Reset password with token
/api/auth/change-password	POST	Change password (authenticated)

7.3.2 Patient APIs

Endpoint	Method	Description
/api/patients/profile	GET	Get patient profile
/api/patients/profile	PUT	Update patient profile
/api/patients/appointments	GET	Get patient's appointments
/api/patients/prescriptions	GET	Get patient's prescriptions
/api/patients/medical-history	GET	Get patient's medical history
/api/patients/reminders	GET	Get patient's reminders

Endpoint	Method	Description
/api/patients/reminders	POST	Create new reminder
/api/patients/reminders/:id	PUT	Update reminder
/api/patients/reminders/:id	DELETE	Delete reminder

7.3.3 Doctor APIs

Endpoint	Method	Description
/api/doctors	GET	Get all doctors (with filters)
/api/doctors/:id	GET	Get doctor details
/api/doctors/profile	GET	Get doctor's own profile
/api/doctors/profile	PUT	Update doctor profile
/api/doctors/availability	GET	Get doctor's availability
/api/doctors/availability	PUT	Update availability slots
/api/doctors/appointments	GET	Get doctor's appointments
/api/doctors/appointments/:id	PUT	Update appointment status
/api/doctors/patients/:patientId	GET	Get patient details
/api/doctors/verify/submit	POST	Submit for verification
/api/doctors/verify/status	GET	Get verification status

7.3.4 Appointment APIs

Endpoint	Method	Description
/api/appointments	POST	Create new appointment
/api/appointments/:id	GET	Get appointment details
/api/appointments/:id	PUT	Update appointment
/api/appointments/:id	DELETE	Cancel appointment
/api/appointments/available-slots	GET	Get available slots for doctor
/api/appointments/check-availability	GET	Check slot availability
/api/appointments/:id/confirm	POST	Confirm appointment
/api/appointments/:id/reschedule	POST	Reschedule appointment

7.3.5 Prescription APIs

Endpoint	Method	Description
/api/prescriptions/upload	POST	Upload prescription
/api/prescriptions/:id	GET	Get prescription details
/api/prescriptions/:id	DELETE	Delete prescription
/api/prescriptions/patient/:patientId	GET	Get patient's prescriptions
/api/prescriptions/doctor/:doctorId	GET	Get prescriptions by doctor
/api/prescriptions/:id/share	POST	Share with doctor
/api/prescriptions/:id/download	GET	Download prescription

7.3.6 Medical History APIs

Endpoint	Method	Description
/api/medical-history/:patientId	GET	Get patient's medical history
/api/medical-history/:id	GET	Get specific record
/api/medical-history	POST	Add medical record
/api/medical-history/:id	PUT	Update medical record
/api/medical-history/:id/export	GET	Export as PDF
/api/medical-history/:patientId/timeline	GET	Get timeline view
/api/medical-history/:patientId/conditions	GET	Get conditions summary

7.3.7 Admin APIs

Endpoint	Method	Description
/api/admin/users	GET	Get all users
/api/admin/users/:id	GET	Get user details
/api/admin/users/:id/block	POST	Block user
/api/admin/users/:id/unblock	POST	Unblock user
/api/admin/doctors/pending	GET	Get pending verifications
/api/admin/doctors/:id/verify	POST	Verify doctor
/api/admin/doctors/:id/reject	POST	Reject verification
/api/admin/stats	GET	Get system statistics
/api/admin/reports/appointments	GET	Appointment report
/api/admin/reports/users	GET	User report
/api/admin/logs	GET	View system logs
/api/admin/settings	GET	Get system settings
/api/admin/settings	PUT	Update system settings

7.3.8 Chatbot APIs

Endpoint	Method	Description
/api/chatbot/query	POST	Send query to chatbot
/api/chatbot/session	POST	Start new session
/api/chatbot/session/:id	GET	Get session history
/api/chatbot/feedback	POST	Submit feedback
/api/chatbot/faq	GET	Get FAQ list

7.3.9 Reminder and Automation APIs

Endpoint	Method	Description
/api/reminders	GET	Get reminders
/api/reminders	POST	Create reminder

Endpoint	Method	Description
/api/reminders/:id	PUT	Update reminder
/api/reminders/:id	DELETE	Delete reminder
/api/reminders/:id/snooze	POST	Snooze reminder
/api/automation/workflows	GET	Get automation workflows
/api/automation/workflows/:id/trigger	POST	Trigger workflow manually
/api/automation/logs	GET	Get automation logs

7.4 Appendix D: Future Enhancements

- **Video Consultation:** Integrated video calling for remote consultations
- **Online Payment Integration:** Secure payment gateway for consultation fees
- **E-Prescription Generation:** Digital prescription creation for doctors
- **Multi-language Support:** Interface in regional languages
- **Mobile Applications:** Native iOS and Android apps
- **AI Disease Prediction:** Predictive analytics for early disease detection
- **Wearable Integration:** Connect with fitness trackers and smartwatches
- **WhatsApp Integration:** Chatbot and reminders via WhatsApp
- **Blockchain Records:** Immutable medical record storage
- **Voice-Based Booking:** Voice-controlled appointment booking
- **Insurance Integration:** Direct insurance claim processing
- **Lab Integration:** Direct lab report fetching
- **Pharmacy Delivery:** Medicine home delivery integration
- **Second Opinion Service:** Specialist second opinion feature
- **Health Records Import:** Import from other healthcare systems
- **Family Accounts:** Manage family members' healthcare
- **Health Analytics:** Advanced health trend analysis
- **Emergency Services:** Direct emergency service integration
- **Telemedicine License:** Multi-region telemedicine compliance
- **Blockchain Consent:** Patient-controlled data access